

Recommender Engine II – Detailed Roadmap

Short description: Build movie recommenders using KNN prediction and K-Means user clustering.

Overall Task

You need to build and evaluate two simple movie recommender systems. Task 1 uses KNeighborsClassifier to predict whether a user likes or dislikes a movie from genre features. Task 2 uses K-Means to cluster similar users and manually recommend movies inside one selected cluster.

Required Final Deliverables

Deliverable	What it must contain
Written report	Results of Task 1 and Task 2, tables, plots, accuracy, explanations, comments.
Python code	Clean, commented code. Clearly mark any code created with generative AI.
Presentation	Show and run the code, explain the tasks, show results, and comment on findings.

Task 1 Roadmap: KNN Movie Recommender

Step 1: Prepare the movie dataset

Open movies-mod-genre-new.csv. Check that every row has a movie name, genre columns, and a rating column. The genre columns are the input features. The rating column is the target label.

Step 2: Rate the movies

Replace or create ratings according to your own taste. Use 1 for movies you like and 0 for movies you dislike. If your group has two users, either create two rating columns or create two separate dataset files.

Step 3: Select input and output columns

Use only the genre columns as X. Use the rating column as y. Keep the movie names separately because you need them later when printing prediction results.

Step 4: Create the unseen dataset

Randomly move about 10% of the movies into a separate unseen dataset. Do not use this data during training or k selection. This unseen part represents new movies that the recommender has not seen before.

Step 5: Use the remaining 90% for modelling

Take the reduced dataset and split it into training and testing data using an 80/20 split. The training part teaches the model. The testing part helps you compare different k values.

Step 6: Try different k values

Run KNeighborsClassifier with several k values, for example k = 1 to 15. For each k, train the model on the training data and predict the test data.

Step 7: Calculate mean error

For every k value, compare predicted ratings with known ratings. Calculate mean error. A lower mean error means better performance for that split.

Step 8: Plot mean error vs k

Create a line plot where the x-axis is k and the y-axis is mean error. The best k is usually the point where mean error is minimum.

Step 9: Repeat k search three times

Repeat the 80/20 split and k testing three times with random sampling. This makes your chosen k more reliable because the result does not depend on only one random split.

Step 10: Choose the final k

Compare the three plots and selected k values. Choose a final k based on the lowest errors and stable performance. Explain your choice in the report.

Step 11: Train the final KNN model

Train KNeighborsClassifier again using the reduced 90% dataset and the selected final k. This gives your final model before unseen testing.

Step 12: Test on unseen movies

Use the trained model to predict ratings for the 10% unseen movies. Print movie name, known rating, predicted rating, and whether the prediction is correct.

Step 13: Calculate final accuracy

Accuracy means correct predictions divided by total predictions. Example: if 5 out of 6 unseen movies are correct, accuracy is $5/6 = 83.33\%$.

Step 14: Repeat final testing five times

Perform the unseen test five times as required. Record the accuracy each time and calculate the overall average accuracy.

Step 15: Comment on the result

Explain whether the KNN recommender worked well or not. Mention possible reasons, such as small dataset size, limited genre features, or subjective ratings.

Task 2 Roadmap: K-Means User Clustering Recommender

Step 1: Collect all user files

Use u1-n.csv, u2-n.csv, ..., u10-n.csv. Each file represents one user and contains movies that user watched.

Step 2: Read each user file

Load each CSV file using pandas. Add a user identifier such as user1, user2, and so on, so you can track which data belongs to which user.

Step 3: Accumulate genre values

For each user, sum the genre columns across all watched movies. For example, if user1 watched three Action movies, the Action value for user1 becomes 3.

Step 4: Build the user-feature table

Create one final table where each row is one user and each column is an accumulated genre value. This table is the input data for K-Means.

Step 5: Save the accumulated table

Save or print the accumulated user-feature table. Include it in your report because the lab task asks for the generated source file.

Step 6: Apply K-Means for different k values

Run KMeans with different numbers of clusters, for example $k = 1$ to 10. For each k , store the SSE/inertia value.

Step 7: Plot SSE vs k

Create a plot where the x-axis is k and the y-axis is SSE. SSE usually decreases when k increases, but you need to find the elbow point.

Step 8: Choose the best k

Use the elbow method. Choose the k where SSE starts decreasing slowly instead of sharply. Explain this decision clearly in your report.

Step 9: Run final K-Means

Run KMeans again using your selected k value. This produces the final clusters.

Step 10: Print clusters and users

For each cluster, print which users belong to it. Example: Cluster 0 contains user1, user4, user7.

Step 11: Choose one cluster

Pick one cluster for manual recommendation. It is better to choose a cluster with at least two users because recommendations need comparison between users.

Step 12: Choose one target user

Inside the selected cluster, choose one user as the target user. This is the user for whom you will recommend movies.

Step 13: Find movies watched by cluster neighbors

Look at movies watched by other users in the same cluster. These users are considered similar to the target user.

Step 14: Remove already watched movies

Compare the target user's watched movies with the cluster group's movies. Remove movies the target user has already watched.

Step 15: Recommend remaining movies

The remaining movies are your recommendation list. Show the selected cluster, selected user, watched movies, candidate movies, and final recommended movies.

Step 16: Comment on the result

Explain whether the recommendations make sense. Mention that this is a simple manual recommendation based on user similarity from genre preferences.

Suggested Report Structure

1. Introduction: Explain the goal of both recommender systems.
2. Dataset description: Explain movie genre data, ratings, and user files.
3. Task 1 method: Explain KNN, data split, k testing, and unseen testing.
4. Task 1 results: Add mean error plots, selected k, prediction table, and accuracy.
5. Task 1 discussion: Comment on accuracy and limitations.
6. Task 2 method: Explain user-feature table and K-Means clustering.
7. Task 2 results: Add accumulated table, SSE plot, chosen k, and clusters.
8. Task 2 recommendation example: Show one cluster, one user, and recommended movies.
9. Conclusion: Compare both approaches and summarize findings.
10. Appendix: Add complete commented Python code.

Simple Work Order

First finish Task 1 completely, because it is more direct: rating, KNN, accuracy. Then finish Task 2: combine user files, cluster users, and create one manual recommendation example. Finally, prepare the report and presentation using your tables, plots, and code outputs.